Interaction Portal (v2)

An interactive, browser-based collaborative research platform for empirical studies on human-AI collaboration.

The **Interaction Portal** places human participants in a collaborative team setting where they work alongside four domain specialist Artificial Intelligence (AI) agents and an AI Orchestrator to solve multi-faceted marketing campaign challenges across seven client scenarios.

All participant-facing wording, study screens, questions, agent prompts, client briefs, and experimental parameters are defined in the `config/` directory. The Python application code only manages the mechanics and data logging.

Overview and Study Design

The platform investigates how different forms of algorithmic support affect team dynamics, task performance, and participant leadership.

Domain Specialist Agents

Participants collaborate with four specialized domain agents:

- **Nia**: Creative Concept and Copywriting. Nia develops campaign themes, messaging strategies, slogans, and creative narratives.
- **Theo**: Audience and Market Research. Theo analyzes consumer demographics, user habits, market segmentation, and competitive positioning.
- **Rhys**: Channels and Content Distribution. Rhys plans social media mix, content calendars, platform engagement, and distribution formats.
- **Mira**: Budget, Feasibility, and Metrics. Mira estimates costs, allocates budgets, defines Key Performance Indicators (KPIs), and evaluates operational feasibility.

AI Orchestrator

An **AI Orchestrator** monitors team conversations, manages conversational pacing, provides dynamic progress feedback through a side panel, and drafts recommendation cards according to experimental conditions.

Experimental Conditions

The platform supports ten between-subject experimental conditions defined in `config/conditions.yaml`:

Condition Code	Orchestrator Role	Orchestrator Panel Level	AI Support Active
`PAS-COORD`	Passive	Coordination	Yes
`PAS-TASK`	Passive	Task-Focused	Yes
`PAS-DEV`	Passive	Developmental	Yes
`EVA-COORD`	Evaluative	Coordination	Yes
`EVA-TASK`	Evaluative	Task-Focused	Yes
`EVA-DEV`	Evaluative	Developmental	Yes
`GEN-COORD`	Generative	Coordination	Yes
`GEN-TASK`	Generative	Task-Focused	Yes
`GEN-DEV`	Generative	Developmental	Yes
`NOAI`	None	None	No (Solo baseline)

- **Passive Role**: The AI Orchestrator observes and responds only when directly addressed. Recommendation cards are left entirely for the human participant to author.
- **Evaluative Role**: The AI Orchestrator evaluates suggestions, identifies gaps, and critiques proposed directions on the recommendation cards.
- **Generative Role**: The AI Orchestrator proactively drafts completed campaign recommendation cards for the participant to inspect, edit, or approve.
- **No-AI Baseline (`NOAI`)**: The participant completes campaign briefs independently without AI agents, establishing an empirical baseline.

Study Flow

Each participant progresses through a structured sequence of stages:

1. **Orientation**: Information sheet, technical checks (viewport width, desktop verification), and consent.
2. **Meet Team**: Introduction to the four domain specialist agents (Nia, Theo, Rhys, Mira) and their responsibilities. In the `NOAI` condition, this screen is automatically skipped.
3. **Leadership Role**: Assignment of the participant's role as team leader and instructions on coordinating team contributions.
4. **Practice Tutorial**: Interactive, hands-on practice step guiding the participant through team messaging, `@mention` routing, panel inspection, and card editing.
5. **Client Tasks**: Collaboration across seven client campaign briefs (six randomized client briefs plus one fixed final client brief). During each client task:
    - Participants review the client brief and private reference pack materials.
    - Participants exchange messages with specialists via broadcast or direct `@mention` calls.
    - An authoritative server countdown timer enforces task duration.
    - An optional rest break is provided between client rounds.
6. **Card Editor and Submission**: Participants complete structured campaign cards across four domains (Creative, Audience, Channels, Budget). The editor tracks typing share and paste operations.
7. **Post-Task Check-In**: A brief reflection scale administered after each client submission.
8. **Exit Survey Hand-off**: Generation of a cryptographically signed completion code and automatic redirection to Qualtrics.

Architecture and Technology Stack

- **Backend**: Python 3.10+ (compatible up to Python 3.13); FastAPI web framework, Starlette, and Uvicorn Asynchronous Server Gateway Interface (ASGI) server.
- **Frontend**: Jinja2 HyperText Markup Language (HTML) templates, vanilla JavaScript (ECMAScript 6+), and vanilla Cascading Style Sheets (CSS) with responsive design and theme support.
- **Data Validation and Configuration**: Pydantic v2 and PyYAML.
- **Database & Telemetry**: SQLite in Write-Ahead Log (WAL) mode with process-wide write locks and append-only audit triggers on event tables.
- **AI Provider Integrations**:
    - Anthropic (Claude)
    - OpenAI (ChatGPT)
    - DeepSeek
    - Zhipu AI (GLM)
    - Alibaba Cloud DashScope (Qwen)
    - Offline Mock Provider (deterministic, zero network calls, zero API costs)

Database Isolation and Modes

The Interaction Portal maintains strict physical database isolation to ensure test activities never contaminate empirical research data:

1. **Standalone Test Mode (`PORTAL_STANDALONE=1`)**:
    - Uses `logs/portal_test.sqlite3`.

- Designed for local trial runs, pilot testing, and researcher exploration via the test launcher.
- Displays a prominent gold test banner on the administrator dashboard.

2. **Live Study Mode (`PORTAL_STANDALONE=0`)**:
   - Uses `logs/portal.sqlite3`.
   - Reserved for real participant sessions arriving via signed entry links from Qualtrics or Prolific.
   - Displays a green live study banner on the administrator dashboard.
3. **Automated Test Suites (`pytest`)**:
   - Use isolated temporary database files in system temporary storage (`tmp_path`).
   - Never write to either `logs/portal.sqlite3` or `logs/portal_test.sqlite3`.

Project Structure

```
interaction_portal/
|--  app/                          # Core application source code
|    |--  agents/                  # Domain specialist agents, orchestrator, and provider clients
|    |    |--  base.py             # Abstract LLM client and provider factories
|    |    |--  mock_llm.py         # Deterministic offline mock provider
|    |    |--  orchestrator.py     # AI Orchestrator decision logic and prompt assembling
|    |    |--  pii.py              # Personally Identifiable Information (PII) scanning and redaction
|    |    \--  specialist.py       # Domain specialist agent execution
|    |--  harnesses/               # Test and evaluation verification harnesses
|    |    |--  harness_a.py        # Static rule and client-asset validation
|    |    \--  harness_b.py        # Autopilot end-to-end simulation driver
|    |--  simulation/              # Participant simulation and state management
|    |    |--  card_provenance.py  # Card editing, diff tracking, and typed-share validation
|    |    |--  chat_router.py      # Message routing, broadcast parsing, and @mention extraction
|    |    |--  practice.py         # Interactive practice tutorial service
|    |    \--  session_manager.py  # Participant session lifecycle and timer enforcement
|    |--  static/                  # Static assets
|    |    |--  css/portal.css      # Responsive styling and CSS custom properties
|    |    \--  js/                 # Client-side controllers (chat, editor, timers)
|    |--  templates/               # Jinja2 HTML templates for each study screen
|    |--  tools/                   # Researcher operational CLI utilities
|    |    |--  backup.py           # Hot database backup utility
|    |    |--  preflight.py        # Pre-flight readiness verification
|    |    |--  purge_pii.py        # Database PII inspection and redaction utility
|    |    \--  stress_test.py      # Concurrent virtual participant stress testing
|    |--  config_loader.py         # Typed configuration parsing and schema validation
|    |--  event_store.py           # Append-only SQLite telemetry store and CSV export generator
|    |--  main.py                  # FastAPI route definitions and application factory
|    |--  state_machine.py         # Experiment flow and progression engine
|    \--  terminal.py              # Cross-platform ANSI terminal coloring helper
|--  config/                       # Application configuration
|    |--  config.yaml              # Main study settings, timers, and server parameters
|    |--  conditions.yaml          # Definitions for all 10 experimental conditions
|    |--  content/clients/         # Client campaign briefs (C1 through C7) and private packs
|    \--  prompts/                 # System prompts for specialists and orchestrator
|--  logs/                         # SQLite databases and runtime log files
|--  tests/                        # Automated unit and integration tests (pytest)
|--  .agents/                      # Antigravity agent configuration, skills, and memory
|--  production_artifacts/         # Shared specifications and deliverables
|--  app_build/                    # Isolated sandbox for newly scaffolded applications
|--  GEMINI.md                     # Antigravity agent project instructions
|--  README.md                     # Human-facing project guide and study documentation
\--  requirements.txt              # Pinned Python package dependencies
```

Quick Start

1. Prerequisites

- **Python 3.10+** (Python 3.10, 3.11, 3.12, or 3.13).
- An isolated virtual environment (recommended): `bash conda activate agents # or: python -m venv .venv && source .venv/bin/activate`
- Install dependencies: `bash pip install -r requirements.txt`

2. Run the Development Server

To launch the portal in standalone test mode with the interactive condition launcher:

```
python -m app.main
```

Or run via Uvicorn directly:

```
python -m uvicorn app.main:app --host 127.0.0.1 --port 8000 --reload
```

Open your browser at: `http://127.0.0.1:8000`

3. Run Automated Tests

Execute the complete test suite using pytest:

```
pytest -q
```

All automated tests use the offline deterministic mock provider and isolated temporary storage. They make zero external network requests and do not pollute the database.

Configuration Guide

The application is fully configurable via YAML files in the `config/` directory.

Main Configuration (`config/config.yaml`)

- `server`: Network host, port (default: 8000), minimum viewport width, session secret key, and maximum concurrent participant capacity.
- `llm`: Provider selection (`mock`, `openai`, `claude`, `deepseek`, `glm`, `qwen`), model names, request timeouts, and temperature settings.
- `timers`: Authoritative countdown durations for client tasks (default: 600 seconds / 10 minutes), rest breaks, and minimum inspection times.
- `panel`: Word budget (default target: 70 words, range 60 to 80 words), minimum refresh interval (60 seconds), and prohibited evaluative terms for coordination panels.
- `editor`: Card submission rules, minimum required typed share, and maximum character length per field.
- `clients`: List of randomized client brief identifiers (e.g., `['C1', 'C2', 'C3', 'C4', 'C5', 'C6']`) and the fixed final client brief (`C7`).

Experimental Conditions (`config/conditions.yaml`)

Defines the 10 experimental conditions. Each condition specifies: * `role`: Orchestrator role (`passive`, `evaluative`, `generative`, `none`). * `panel_level`: Side panel feedback detail (`coordination`, `task_focused`, `developmental`, `none`). * `ai`: Boolean flag. When `false`, removes chat, panels, and AI specialist team members.

Environment Variables and Configuration Precedence

The Interaction Portal uses a three-tier configuration hierarchy to resolve settings at runtime.

Configuration Precedence Hierarchy

Settings are resolved in the following order of priority:

1. **Tier 1 (Highest Priority): Command-Line and Process Environment Variables**: Variables defined directly in the shell or process environment take precedence over all other sources. They override file settings for the duration of the current run without permanently modifying system settings.
2. **Tier 2 (Intermediate Priority): Local `.env` File**: If a `.env` file exists in the repository root or in the `config/` directory, the application loads its variables on startup. The loader only populates variables that are not already present in `os.environ`. This guarantees that command-line overrides are never replaced. A documented template is provided in `.env.example`.

3. **Tier 3 (Base Defaults): YAML Configuration**: When a variable is neither defined in the process environment nor in a `.env` file, the application falls back to the defaults defined in `config/config.yaml`.

Setting Environment Variables Across Platforms

Depending on your operating system and command shell, use the following syntax to set environment variables for the current run or terminal session.

1. Windows PowerShell

- **Run with variable for a single execution**: `powershell env:PORTAL_PORT="8001"; python -m app.main`
- **Set variable for the active PowerShell session**: `powershell env:PORTAL_PORT = "8001" python -m app.main`
- **Clear or reset the variable**: `powershell Remove-Item env:PORTAL_PORT`

2. Windows Command Prompt (`cmd.exe`)

- **Run with variable for a single execution**: `cmd set PORTAL_PORT=8001 && python -m app.main`
- **Set variable for the active Command Prompt session**: `cmd set PORTAL_PORT=8001 python -m app.main`
- **Clear or reset the variable**: `cmd set PORTAL_PORT=`

3. Linux and macOS (Bash / Zsh)

- **Inline execution for a single command**: `bash PORTAL_PORT=8001 python -m app.main`
- **Set variable for the active shell session**: `bash export PORTAL_PORT=8001 python -m app.main`
- **Clear or reset the variable**: `bash unset PORTAL_PORT`

Environment Variables Reference Table

The following table documents all environment variables recognized by the Interaction Portal, their corresponding path in `config/config.yaml`, default values, and operational purpose.

Variable Name	`config.yaml` Mapping	Default Value	Description
`PORTAL_STANDALONE`	`testing.standalone_mode`	`1` (true)	Set to `1` for standalone test mode with interactive launcher at `/`, or `0` for production mode requiring signed entry links.
`PORTAL_ENV`	N/A	`development`	Operational environment (`development`, `testing`, `production`). In production, checks for default tokens and raises an error if insecure defaults are used.
`PORTAL_HOST`	`server.host`	`127.0.0.1`	Network interface address to which the server binds.
`PORTAL_PORT`	`server.port`	`8000`	Transmission Control Protocol (TCP) port number on which the web server listens.
`PORTAL_SECRET_KEY`	`server.secret_key`	`change-me-in-production`	Secret cryptographic key used to sign and verify browser session cookies.
`PORTAL_ADMIN_TOKEN`	`testing.admin_token`	`admin-test-token`	Bearer authentication token required to query administrative export endpoints (`/admin/export/*`).
`PORTAL_COMPLETION_SECRET`	`qualtrics.completion_code_s`	`change-me-too`	Hash-based Message Authentication Code (HMAC) secret used to generate and verify participant exit completion codes. Also accepts `PORTAL_COMPLETION_CODE_SECRET`.

Variable Name	`config.yaml` Mapping	Default Value	Description
`PORTAL_ENTRY_SECRET`	Inferred	Defaults to `PORTAL_SECRET_KEY`	Cryptographic key used to validate participant entry link signatures in production mode.
`PORTAL_REQUIRE_ENTRY_SIG`	Inferred	`true` in production, `false` in standalone	Enforces signature verification on incoming participant entry links.
`PORTAL_COOKIE_SECURE`	Inferred	`false` in development, `true` in production	Requires the `Secure` flag on session cookies, requiring HyperText Transfer Protocol Secure (HTTPS).
`PORTAL_MAX_PARTICIPANTS`	`server.max_concurrent_participants`	`100`	Maximum number of active participant sessions permitted concurrently.
`PORTAL_TIMER_SCALE`	`testing.timer_scale`	`1.0`	Accelerator factor for all task timers and reading delay checks. A value of `0.05` accelerates timers by a factor of 20 for rapid testing.
`PORTAL_CONFIG_DIR`	N/A	`config/`	Path to a custom directory containing configuration YAML and prompt markdown files.
`PORTAL_DB_PATH`	N/A	`logs/portal_test.sqlite3` (test) or `logs/portal.sqlite3` (prod)	Explicit file path for the SQLite database. Overrides standard test and live database file paths.
`PORTAL_SKIP_STARTUP_TESTS`	N/A	`0`	When set to `1`, bypasses the automated self-check test suite on server startup.
`WEB_CONCURRENCY`	N/A	Unset (`1`)	Process worker count. Must remain `1` or unset because the portal coordinates in-memory state and locks within a single process.
`PORTAL_LLM_PROVIDER`	`llm.provider`	`mock`	Selected model provider: `mock`, `openai`, `claude`, `deepseek`, `glm`, or `qwen`.
`PORTAL_LLM_MODEL`	`llm.model`	Provider default	Pinned Large Language Model (LLM) name. Defaults adapt to provider (for example `gpt-5.4-mini`, `claude-haiku-5-5`, `deepseek-flash`, `glm-5.3-flash`, `qwen3.8-flash`).
`PORTAL_LLM_BASE_URL`	`llm.base_url`	Provider default	Custom Application Programming Interface (API) endpoint Uniform Resource Locator (URL) for OpenAI-compatible gateways.
`PORTAL_LLM_API_KEY_ENV`	`llm.api_key_env`	Provider default	Name of the environment variable that supplies the provider API key.
`PORTAL_LLM_API_KEY`	N/A	None	Universal API key override across any active model provider.

Variable Name	`config.yaml` Mapping	Default Value	Description
`OPENAI_API_KEY`	N/A	None	Primary API authentication key for OpenAI / ChatGPT models (fallback: `CHATGPT_API_KEY`).
`ANTHROPIC_API_KEY`	N/A	None	Primary API authentication key for Anthropic Claude models (fallback: `CLAUDE_API_KEY`).
`DEEPSEEK_API_KEY`	N/A	None	Primary API authentication key for DeepSeek models.
`ZAI_API_KEY`	N/A	None	Primary API authentication key for GLM / Z.ai models (fallbacks: `GLM_API_KEY`, `ZHIPUAI_API_KEY`, `BIGMODEL_API_KEY`).
`DASHSCOPE_API_KEY`	N/A	None	Primary API authentication key for Qwen / Alibaba Cloud DashScope models (fallback: `QWEN_API_KEY`).
`NO_COLOR`	N/A	None	When set to any value, disables colored ANSI terminal output in the console.
`FORCE_COLOR`	N/A	None	When set to `1`, forces colored ANSI terminal output even in non-interactive terminals.

Troubleshooting

Problem	Root Cause	Solution
`Port 8000 is already in use`	Another server instance or application is using port 8000.	Change port in `config/config.yaml` under `server.port`, or run with `env:PORTAL_PORT="8001"; python -m app.main` (PowerShell) or `PORTAL_PORT=8001 python -m app.main` (Linux/macOS).
`Insecure server.secret_key in production`	Production mode was started with the default secret key.	Set `PORTAL_SECRET_KEY` to a random secret of at least 24 characters before starting the server.
`Insecure admin_token in production`	Production mode was started with the default admin token.	Set `PORTAL_ADMIN_TOKEN` to a secret password of at least 24 characters before starting the server.
`AI Provider 401 Unauthorized`	Missing or invalid API key for the chosen LLM provider.	Verify the appropriate environment variable is set (for example `ANTHROPIC_API_KEY` or `OPENAI_API_KEY`). For offline testing, set `PORTAL_LLM_PROVIDER=mock`.
`Single-process execution error (WEB_CONCURRENCY > 1)`	Uvicorn or Gunicorn started with multiple worker processes.	The portal manages in-memory participant state and locks in a single process. Set `WEB_CONCURRENCY=1` or leave it unset.
`403 Forbidden on Entry Link`	Entry link signature verification failed or PID format is invalid.	In production, entry links must be signed with `PORTAL_ENTRY_SECRET`. In test mode, set `PORTAL_STANDALONE=1` to disable link signature requirements.
`Missing config text: pages....`	A text key was removed or renamed in `config/config.yaml`.	Run `python -m app.tools.preflight` to identify the missing configuration key.

Practical Guide for Study Hosts

This section explains operational procedures for researchers and study coordinators hosting the **Interaction Portal**.

---

1. Automatic Startup Self-Check in the Console

Whenever you start the portal with `python -m app.main`, the application automatically executes an internal self-check before opening the web interface:

- **Green Checkmark (`[OK] Startup self-check: 14/14 test suites passed`)**: Everything is verified and ready for participants.
- **Orange Warning (`[!] Startup warning: ...`)**: If a configuration file or directory has an issue, the console prints an explanatory warning showing the check that failed and a recommendation to resolve it. **The portal will still start** so ongoing data collection is not interrupted.
- **Optional Skip**: To skip this check on startup, set `PORTAL_SKIP_STARTUP_TESTS=1` in your `.env` file, or run `env:PORTAL_SKIP_STARTUP_TESTS="1"; python -m app.main` (PowerShell) or `PORTAL_SKIP_STARTUP_TESTS=1 python -m app.main` (Linux/macOS).

---

2. Managing Concurrent Participant Capacity (`max_concurrent_participants`)

- **Configuration**: In `config/config.yaml` under `server:`, `max_concurrent_participants` defines the maximum number of concurrent active participants (default: `100`). You can also override this with `PORTAL_MAX_PARTICIPANTS=100`.
- **Capacity Behavior**: If the limit is reached, additional participants attempting to join see a polite waiting screen asking them to refresh momentarily once a slot becomes available. The server does not crash or invalidate their link.
- **Rate Limits Consideration**: With live AI providers, each participant generates multiple LLM calls. Ensure your external AI provider account tier allows sufficient Requests Per Minute (RPM) for your target concurrent wave size.

---

3. Running a Simulation Stress Test (`stress_test.py`)

Before inviting paid participants, verify your server hardware and network using the built-in stress testing tool:

```
# Test 30 concurrent participants using the offline mock model
python -m app.tools.stress_test --participants 30 --llm-delay-ms 500

# Test 100 concurrent participants
python -m app.tools.stress_test --participants 100 --llm-delay-ms 500
```

Verify that the report concludes with `Stress Test Verdict: PASSED` and `Participants Completed: 100.0%`.

---

4. Participant Session Recovery

- **Same Browser within 15 Minutes (Automatic Recovery)**: If a participant accidentally closes their tab, refreshes, or briefly loses internet connectivity, they can reopen their original link in the same browser within 15 minutes (`timers.resume_gap_seconds: 900`). The server restores their exact screen and remaining timer state via secure session cookies.
- **Different Browser or Cleared Cookies (Manual Host Recovery)**: If a participant switches devices or cleared their browser cookies, the server blocks re-entry under the raw Participant Identifier (PID) to prevent session collisions. The study host can provide a recovery link:
  1. Open the administrator dashboard at `http://127.0.0.1:8000/admin?token=admin-test-token`.
  2. Locate the participant's PID in the **Recent Sessions** table.
  3. Copy the **Resume URL** (`http://127.0.0.1:8000/?resume=...`) from the **Recovery** column and provide it to the participant.

---

5. Pre-Flight Readiness Verification (`preflight.py`)

Run the pre-flight verification script to ensure all assets, conditions, briefs, and database files are intact:

```
python -m app.tools.preflight
```

Ensure the output concludes with `Verdict: READY`.

---

6. Administrator Dashboard and Flat Data Exports

Access the administrator dashboard in your web browser: `http://127.0.0.1:8000/admin?token=admin-test-token`

(In production, replace `admin-test-token` with the secret configured in `PORTAL_ADMIN_TOKEN`.)

Features Available in the Admin Dashboard:

- **Active Mode Banner**: Displays whether the administrator is viewing the Test Database (`portal_test.sqlite3`) or the Production Database (`portal.sqlite3`).
- **Study Overview**: Displays Total Sessions, Completed Sessions, Active Sessions, and Dropped Sessions against current capacity.
- **Condition Balance Table**: Shows assignment and completion counts across all 10 experimental conditions.
- **Flat Data Exports**: One-click downloads of researcher-facing Comma-Separated Values (CSV) datasets:
  - **Task Summary (Wide CSV)** (`/admin/export/task_summary.csv`): Primary dataset for statistical analysis. One row per participant per client task, including message counts, `@mention` counts, hidden items uncovered, panel views, card typed share, and check-in scores.
  - **Events CSV** (`/admin/export/events.csv`): Complete chronological event log.
  - **Panels CSV** (`/admin/export/panels.csv`): Word counts, sections, and validation history for orchestrator panels.
  - **Transcripts CSV** (`/admin/export/transcripts.csv`): Full message transcripts for all conversations.
  - **Card Spans CSV** (`/admin/export/card_spans.csv`): Detailed text span provenance and typing analytics.
  - **Check-ins CSV** (`/admin/export/checkins.csv`): Post-task reflection responses.
  - **Sessions CSV** (`/admin/export/sessions.csv`): High-level session metadata, start/end timestamps, and completion codes.
  - **Full JSON Dump** (`/admin/export/all.json`): Comprehensive raw database export.

---

7. Database Backups and PII Redaction

- **Hot Database Backups**: Create an immediate, non-blocking backup of the active database while participants are actively working: `bash python -m app.tools.backup --dest backups/ --keep 14`
- **Personally Identifiable Information (PII) Redaction**: Inspect and redact any inadvertently typed email addresses, phone numbers, or personal names from chat transcripts and card submissions: ```bash # Preview detected personal details without altering data python -m app.tools.purge_pii --dry-run

Apply redactions across all tables python -m app.tools.purge_pii ```